

华东师范大学数据科学与工程学院实验报告

课程名称: AI 基础	年级: 2024	上机实践时间: 2026.06.27
指导教师: 杨彬、树杨	姓名: 孙育泉	
上机实践名称: Project 2	学号: 10234900421	

I 背景介绍

本项目选择 Kaggle 比赛 Child Mind Institute - Detect Sleep States 作为实验对象，完成一个基于手环传感器时间序列的睡眠事件检测系统。虽然比赛输出不是未来连续数值序列，而是离散的 onset 和 wakeup 事件点，但其本质仍然是时间序列建模问题：模型需要根据历史和局部上下文中的传感器变化，判断每个 step 是否接近入睡或醒来事件。因此，我将任务建模为逐时间步的 dense sequence prediction，再通过后处理把概率序列转换为离散事件提交结果。

II 数据集与任务分析

II.1 数据来源

实验数据来自 Kaggle 比赛 Child Mind Institute - Detect Sleep States。该比赛要求根据儿童佩戴手环采集到的传感器序列，自动检测睡眠过程中的两个关键事件：

- (1) onset：入睡时刻。
- (2) wakeup：醒来时刻。

数据文件主要包括：

文件	含义
train_series.parquet	训练集传感器时间序列。
train_events.csv	训练集事件标注，包含 series_id、night、event、step、timestamp 等字段。
test_series.parquet	测试集传感器时间序列。
sample_submission.csv	提交格式样例，列为 row_id, series_id, step, event, score。

II.2 核心字段说明

四个数据文件中，最关键的是 train_series.parquet、train_events.csv 和 test_series.parquet 中的时间序列字段与事件字段。

字段名	含义
series_id	一条连续传感器记录的编号。模型训练和验证时需要按 series_id 分组，避免同一条时间序列同时出现在训练集和验证集中。
step	时间步编号。比赛数据中 1 个 step 对应 5 秒，因此可以通过 step 推算事件在时间轴上的相对位置。最终提交时也需要给出预测事件对应的 step。
timestamp	真实时间戳。该字段可以用于构造小时、分钟、星期等周期性时间特征，因为睡眠事件与一天中的时间有明显关系。

anglez	手腕角度传感器特征, 反映佩戴者手腕姿态.睡眠状态下手腕角度通常较稳定, 而醒来或活动时角度变化会更明显.
enmo	运动强度特征, 反映佩戴者的身体活动程度.入睡后运动强度通常降低, 醒来前后运动强度往往会上升, 因此它是判断睡眠事件的重要输入.
night	train_events.csv 中的字段, 表示同一条 series_id 中第几个夜晚的标注.它帮助理解一条长时间序列中可能包含多个睡眠周期.
event	train_events.csv 中的字段, 表示事件类型, 只有 onset 和 wakeup 两类.onset 表示入睡时刻, wakeup 表示醒来时刻.
score	sample_submission.csv 和最终提交文件中的字段, 表示模型对某个预测事件的置信度.Kaggle 会按照该分数从高到低计算事件平均精度.

II.3 任务难点

本任务与普通分类任务相比有以下难点:

- (1) 时间序列很长.单条 series 可能覆盖多天数据, 不能把每一行当作独立样本处理.
- (2) 事件极其稀疏.绝大多数 step 不是 onset 或 wakeup, 正负样本比例严重不平衡.
- (3) 标注存在缺失.部分夜晚的事件 step 为 NaN, 训练时需要过滤无效标注.
- (4) 评价指标具有时间容忍.预测事件只要落在真实事件附近的一定时间窗口内就可能被视为正确, 因此模型不一定要预测单个精确点, 而应尽量形成稳定峰值.
- (5) 模型输出与提交格式不同.神经网络通常输出每个 step 的概率, 而 Kaggle 需要离散事件列表, 因此后处理非常关键.

II.4 评价指标

本比赛的评价指标不是普通分类准确率, 而是面向事件检测任务的 Average Precision.原因是模型最终提交的是若干个离散事件点, 每个预测事件都有一个置信度 score.评价时会按照置信度从高到低依次匹配真实事件, 并计算 Precision-Recall 曲线下的面积.

设真实事件集合为:

$$G = \{g_i\}_{i=1}^N, \quad g_i = (s_i, e_i, t_i)$$

其中 s_i 表示 series_id, $e_i \in \mathcal{E}$ 表示事件类型, t_i 表示真实事件发生的 step.事件类型集合为:

$$\mathcal{E} = \{\text{onset}, \text{wakeup}\}$$

对于某一类事件 e , 真实事件子集记为:

$$G_e = \{g_i \in G \mid e_i = e\}$$

模型提交的预测事件集合为:

$$P = \{p_j\}_{j=1}^M, \quad p_j = (\hat{s}_j, \hat{e}_j, \hat{t}_j, c_j)$$

其中 $\hat{s}_j$ 是预测事件所属的 series_id, $\hat{e}_j$ 是预测事件类型, $\hat{t}_j$ 是预测事件 step, $c_j \in [0, 1]$ 是模型给出的置信度分数.对某一类事件 e , 预测子集为:

$$P_e = \{p_j \in P \mid \hat{e}_j = e\}$$

评价时，首先将同一事件类型下的预测按照置信度从高到低排序：

$$c_1 \geq c_2 \geq \dots \geq c_M$$

对于给定的时间容忍阈值 τ ，预测事件 p_j 可以匹配真实事件 g_i 的条件为：

$$\hat{s}_j = s_i \quad \wedge \quad \hat{e}_j = e_i \quad \wedge \quad |\hat{t}_j - t_i| \leq \tau$$

也就是说，预测事件必须满足三点：属于同一条时间序列、事件类型相同，并且预测 step 与真实 step 的距离不超过容忍阈值 τ 。

同时，每个真实事件最多只能被匹配一次：

$$\forall g_i \in G, \quad g_i \text{ can be matched at most once}$$

因此，如果多个预测事件都落在同一个真实事件附近，只有置信度排序中最先匹配成功的预测会被记为 True Positive，其余重复预测会被记为 False Positive。这一点很重要，因为它会惩罚模型在同一真实事件附近生成过多重复峰值。

对于排序后的前 k 个预测，定义：

$$\text{TP}(k) = \sum_{j=1}^k \mathbb{I}(p_j \text{ is TP}) \quad \text{FP}(k) = \sum_{j=1}^k \mathbb{I}(p_j \text{ is FP})$$

其中 $\mathbb{I}(\cdot)$ 是指示函数，条件成立时取 1，否则取 0。于是 Precision 和 Recall 分别定义为：

$$\text{Precision}(k) = \frac{\text{TP}(k)}{\text{TP}(k) + \text{FP}(k)}$$

$$\text{Recall}(k) = \frac{\text{TP}(k)}{|G_e|}$$

Precision 衡量当前已经提交的预测中有多少是正确事件；Recall 衡量所有真实事件中有多少已经被找出。

Average Precision 计算的是 Precision-Recall 曲线下的面积。实际实现中通常使用插值后的 precision envelope。对于事件类型 e 和容忍阈值 τ ，AP 定义为：

$$\text{AP}(e, \tau) = \int_0^1 \text{Prec}_e(r; \tau) \, dr$$

离散情况下可以写成：

$$\text{AP}(e, \tau) = \sum_{k=1}^M [\text{Recall}(k) - \text{Recall}(k-1)] \cdot \max_{\tilde{k} \geq k} \text{Precision}(\tilde{k})$$

其中 $\max_{\tilde{k} \geq k} \text{Precision}(\tilde{k})$ 表示从当前 rank 之后能达到的最大 precision，用于得到单调不增的插值 precision 曲线。这样可以避免 Precision-Recall 曲线局部抖动对 AP 产生不合理影响。

本比赛会在多个时间容忍阈值上计算 AP。代码中使用的 tolerance step 为：

$$\mathcal{T} = \{12, 36, 60, 90, 120, 150, 180, 240, 300, 360\}$$

由于比赛数据中 1 step = 5 秒，因此这些阈值大约对应 1 分钟、3 分钟、5 分钟、7.5 分钟、10 分钟、12.5 分钟、15 分钟、20 分钟、25 分钟和 30 分钟。

最终分数是对两类事件和所有时间容忍阈值的 AP 取平均：

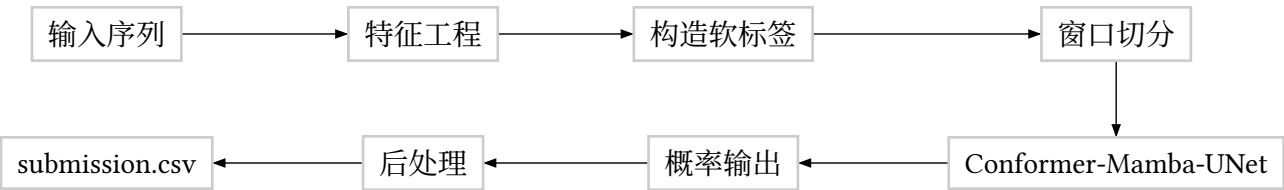
$$\text{Score} = \frac{1}{|\mathcal{E}| \cdot |\mathcal{T}|} \sum_{e \in \mathcal{E}} \sum_{\tau \in \mathcal{T}} \text{AP}(e, \tau)$$

在本项目中，src/metric.py 实现了与上述定义一致的本地近似评分函数.它会先删除无效标注，然后分别对 onset 和 wakeup 计算不同 tolerance 下的 AP，最后取平均作为本地验证分数。

III 整体方案

III.1 方法总览

本项目采用的整体流程如下：



模型不是直接输出一个夜晚的入睡和醒来时间，而是对窗口内每个 step 输出四个 logit：

- (1) onset：当前位置接近入睡事件的概率.
- (2) wakeup：当前位置接近醒来事件的概率.
- (3) sleep：当前位置处于睡眠区间的概率.
- (4) invalid：当前位置属于无效或低质量片段的概率.

其中 onset 和 wakeup 是最终提交最直接使用的两个事件通道，sleep 和 invalid 用于辅助训练和后处理。

IV 特征工程

IV.1 基础差分特征

代码 src/features.py 中的基础输入来自 anglez 和 enmo.其中 anglez 描述手腕角度，enmo 描述加速度运动强度.为了增强模型对运动变化的感知，我在基础特征上构造了以下差分和变换特征：

特征	作用
anglez	原始手腕角度.
enmo	原始运动强度，并对负值进行截断.
log1p_enmo	压缩运动强度的长尾分布.
d_anglez	相邻 step 之间角度变化.
abs_d_anglez	角度变化幅度，弱化方向影响.
d_enmo	相邻 step 之间运动强度变化.
abs_d_enmo	运动强度变化幅度.

睡眠期间通常运动强度较低、姿态变化较少，而入睡和醒来附近会出现运动模式变化.因此差分特征对事件边界定位很重要。

IV.2 窗口滚动特征

仅看单个 step 难以判断其睡眠状态，因此项目加入多尺度 rolling 特征.默认窗口为：

text

```
1 12, 60, 360, 1440 steps
```

由于每个 step 为 5 秒, 这些窗口大致对应 1 分钟、5 分钟、30 分钟和 2 小时. 每个窗口计算 anglez 和 enmo 的均值与标准差, 例如:

```
1 anglez_mean_12, anglez_std_12
2 enmo_mean_60, enmo_std_60
3 anglez_mean_360, enmo_std_1440
```

text

这些特征让模型同时看到短时运动变化和长时间趋势. 比如一个短时动作可能只是翻身, 但若长窗口内运动强度持续下降, 就更可能接近入睡.

IV.3 时间周期特征

睡眠事件与一天中的时间高度相关, 所以我加入周期性时间特征:

```
1 hour_sin, hour_cos
2 minute_sin, minute_cos
3 weekday_sin, weekday_cos
```

text

使用 sin/cos 编码的原因是时间具有周期性. 例如 23:55 和 00:05 在数值上相差很大, 但实际时间上非常接近. 周期编码能避免直接使用小时数带来的断点问题.

IV.4 数据质量特征与归一化

代码中还构造了 non_wear_flag 和 repeating_flag. 其中 non_wear_flag 通过 360 step 窗口内的 anglez 和 enmo 标准差判断是否可能为非佩戴片段. 所有非 flag、非周期特征使用 median 和 IQR 做稳健归一化, 并裁剪到 $[-10, 10]$. 这样可以降低异常值对训练的影响.

V 标签设计

V.1 离散标签的问题

原始事件标注是单个 step. 例如某一行表示 step=4992 是 onset. 如果直接构造 one-hot 标签, 则只有极少数 step 为 1, 其余全部为 0. 这样会带来两个问题:

- (1) 训练信号过于稀疏, 模型容易学习到“全部预测为非事件”.
- (2) 评价指标允许一定时间误差, 但 one-hot 标签却只奖励单个精确位置, 与指标不完全匹配.

V.2 高斯软标签

因此, 本项目将每个真实事件扩展为高斯软标签. 距离事件越近, 标签值越接近 1; 距离越远, 标签逐渐衰减到 0. 代码 src/labels.py 中默认使用多个 sigma 叠加:

```
1 sigma_steps = [36, 120, 240]
2 sigma_weights = [0.5, 0.3, 0.2]
```

text

换算成时间, 大致对应 3 分钟、10 分钟和 20 分钟的尺度. 多尺度软标签一方面保留事件中心的强监督, 另一方面给事件附近较宽区域提供较弱监督, 使模型更容易学习入睡和醒来前后的连续变化模式.

V.3 睡眠区间辅助标签

除了 onset 和 wakeup 事件标签, 代码还根据成对的 onset $\rightarrow$ wakeup 标注生成 sleep 区间标签. 若某段 step 位于入睡和醒来之间, 则 $y_{sl} = 1$. 这个辅助任务能帮助模型理解整体睡眠结构, 而不是只在两个事件点附近学习局部峰值.

VI 模型结构

VI.1 输入输出

模型输入是一个固定长度窗口内的特征矩阵:

```
1 shape = [batch_size, window_size, feature_dim]
```

text

主要实验中 $\text{window_size}=7200$, 即 7200 个 step. 由于 $1 \text{ step} = 5 \text{ 秒}$, 一个窗口覆盖 10 小时, 基本可以覆盖一段典型睡眠过程.

模型输出为:

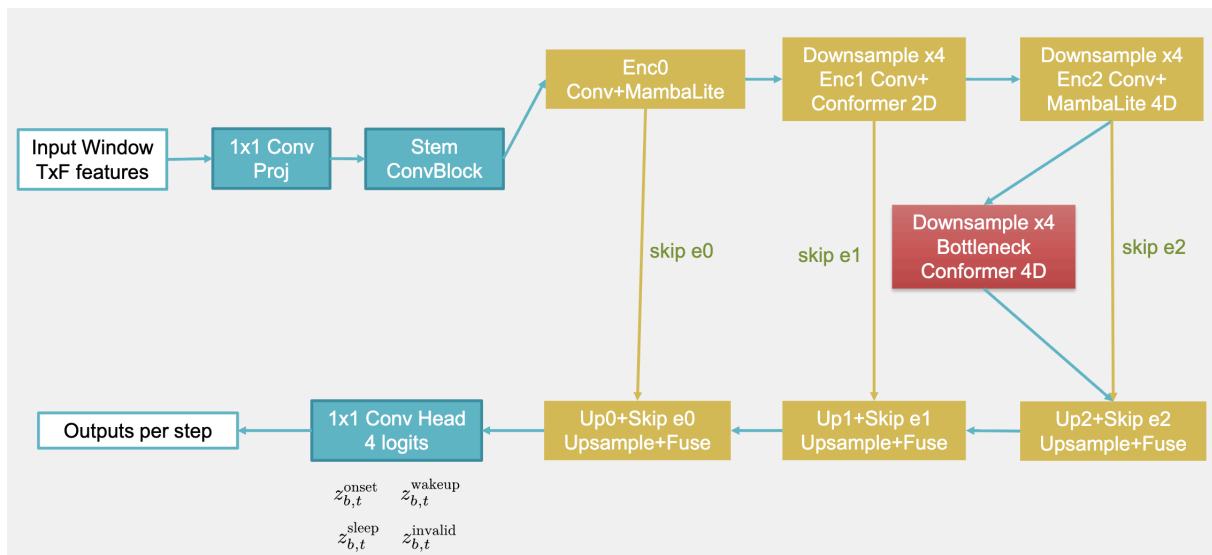
```
1 onset, wakeup, sleep, invalid
```

text

每个输出都是长度为 window_size 的序列, 表示每个 step 对应的 logit.

VI.2 Sleep-Mamba-UNet

本项目核心模型为 SleepMambaUNet. 它是一维 U-Net 风格结构, 结合卷积、Conformer 和 Mamba-like 序列混合模块. 整体结构可以概括为:



U-Net 的优势是兼顾局部细节和长程上下文. 下采样扩大感受野, 适合捕捉睡眠状态的长时间变化; 上采样和 skip connection 则保留事件边界附近的时间定位信息.

VI.3 ConvBlock

ConvBlock 使用 depthwise 1D convolution、pointwise convolution、GLU、BatchNorm、GELU 和 dropout, 并通过残差连接保留原始表示. 它主要负责提取局部运动模式, 例如短时翻身、静止和运动强度突变.

VI.4 Conformer 模块

Conformer 结合自注意力和卷积, 既能建模较长距离依赖, 也能保留局部时序结构. 在本项目中, 它用于 encoder 和 bottleneck 中较低时间分辨率的特征, 降低计算量的同时扩大上下文范围.

VI.5 MambaLite 模块

代码中的 MambaLiteBlock 是一个轻量级状态空间风格序列混合模块.它使用 LayerNorm、线性投影、depthwise convolution、门控和累积混合来近似长程信息传播.相比标准注意力,它更轻量,适合长窗口时间序列.

VI.6 模型设计动机

选择混合结构而不是单一 CNN 或 Transformer 的原因是:

- (1) 睡眠事件有明显局部边界, 需要卷积捕捉短时变化.
- (2) 睡眠周期持续数小时, 需要模块具备较长上下文建模能力.
- (3) Kaggle notebook 资源有限, 不能使用过重的全注意力模型.
- (4) U-Net 的多尺度结构适合把粗粒度睡眠状态和细粒度事件定位结合起来.

VII 训练策略

VII.1 窗口切分

完整 series 很长, 无法直接整段输入模型, 因此代码在 src/data.py 中使用固定窗口切分.主要实验设置为:

```
1 window_size: 7200
2 stride: 7200
3 batch_size: 1
4 folds: 2
```

yaml

窗口不足长度时进行 padding, 并用 mask_valid 标记有效位置.推理阶段使用滑动窗口, 对重叠位置进行平均, 最终得到每个 step 的概率.

VII.2 Group Fold 划分

交叉验证按照 series_id 分组划分, 保证同一条 series 不会同时出现在训练集和验证集中.因为同一受试者或记录内部高度相关, 如果随机按行划分, 会造成信息泄露, 使验证分数虚高.

VII.3 Event-focused Sampling

由于事件非常稀疏, 随机采样窗口会产生大量没有事件的负样本窗口.为提高训练效率, 我在 _limit_windows 中实现了 event-focused sampling: 优先选择 y_onset 或 y_wakeup 峰值较高的窗口, 同时保留一部分均匀分布窗口.

典型配置为:

```
1 window_sampling: event_focused
2 event_window_fraction: 0.8
3 max_windows_per_series: 10
```

yaml

这样可以让模型在有限训练预算下更频繁地看到入睡和醒来片段.

VII.4 损失函数

本任务的核心困难是事件极其稀疏.对于一条很长的时间序列, 绝大多数 step 都不是 onset 或 wakeup.如果直接使用普通二分类交叉熵, 模型很容易被大量负样本主导, 从而倾向于输出很低的事件概率.因此, 本项目的 loss function 不是单一损失, 而是由事件检测损失、睡眠区间辅助损失和边界一致性损失共同组成.

模型对每个 batch 中第 b 个样本、第 t 个时间步输出四个 logit:

$$z_{b,t}^{\text{on}}, z_{b,t}^{\text{wu}}, z_{b,t}^{\text{sl}}, z_{b,t}^{\text{inv}}$$

其中 "on" 表示 onset, "wu" 表示 wakeup, "sl" 表示 sleep, "inv" 表示 invalid. 对应概率由 sigmoid 函数得到:

$$p_{b,t}^{\text{on}} = \sigma(z_{b,t}^{\text{on}}), \quad p_{b,t}^{\text{wu}} = \sigma(z_{b,t}^{\text{wu}})$$

$$p_{b,t}^{\text{sl}} = \sigma(z_{b,t}^{\text{sl}}), \quad p_{b,t}^{\text{inv}} = \sigma(z_{b,t}^{\text{inv}})$$

其中:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

训练标签包括事件软标签 $y_{b,t}^{\text{on}}$ 、 $y_{b,t}^{\text{wu}}$, 睡眠区间标签 $y_{b,t}^{\text{sl}}$, 以及若干 mask.mask_event 用于表示该位置是否参与事件损失, mask_sleep 用于表示该位置是否参与睡眠区间损失, mask_valid 用于表示该位置是否是真实时间步而不是 padding.

VII.4.1 Binary Cross Entropy

基础二分类损失为 BCE. 对于单个预测概率 p 和标签 y , 定义为:

$$\mathcal{L}_{\text{BCE}}(p, y) = -y \log(p) - (1 - y) \log(1 - p)$$

如果直接用于事件检测, 则事件损失可以写为:

$$\mathcal{L}_{\text{event}} = \mathcal{L}_{\text{on}} + \mathcal{L}_{\text{wu}}$$

其中:

$$\mathcal{L}_{\text{on}} = \frac{\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{event}} \mathcal{L}_{\text{BCE}}(p_{b,t}^{\text{on}}, y_{b,t}^{\text{on}})}{\max(\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{event}}, 1)}$$

$$\mathcal{L}_{\text{wu}} = \frac{\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{event}} \mathcal{L}_{\text{BCE}}(p_{b,t}^{\text{wu}}, y_{b,t}^{\text{wu}})}{\max(\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{event}}, 1)}$$

其中 $m_{b,t}^{\text{event}} \in \{0, 1\}$ 是事件 mask. 分母使用 $\max(\dots, 1)$ 是为了避免有效位置数为 0 时出现除零问题.

VII.4.2 Focal Loss

由于事件标签高度不平衡, 大量位置都是简单负样本. 普通 BCE 会让这些简单负样本占据主要梯度, 导致模型对事件点不敏感. 因此后续实验中引入 focal loss.

对于二分类任务, focal loss 可以写成:

$$\mathcal{L}_{\text{Focal}}(p, y) = -\alpha y (1 - p)^\gamma \log(p) - (1 - \alpha) (1 - y) p^\gamma \log(1 - p)$$

其中 α 用于调节正负样本权重, γ 用于降低简单样本的权重.

当 $y = 1$ 且模型已经预测 p 很高时, $(1 - p)^\gamma$ 很小, 该样本损失会被降低; 当 $y = 0$ 且模型预测 p 很低时, p^γ 很小, 该负样本损失也会被降低. 这样模型会更加关注预测错误或不确定困难的样本.

在 event_plus 配置中使用:

```
1 event_loss: focal
```

yaml


```

2 focal_gamma: 1.5
3 focal_alpha: 0.65
4 event_weight: 1.5

```

其中 $\text{focal_alpha} = 0.65$ 表示相对提高正事件区域的权重, $\text{focal_gamma} = 1.5$ 控制困难样本聚焦程度, $\text{event_weight} = 1.5$ 进一步加强事件检测损失在总损失中的比重.

VII.4.3 睡眠区间辅助损失

除了预测 onset 和 wakeup, 模型还输出 sleep 通道.sleep 标签根据成对的 onset $\rightarrow$ wakeup 事件构造: 若某个 step 位于入睡和醒来之间, 则:

$$y_{b,t}^{\text{sl}} = 1$$

否则为 0.睡眠区间损失同样使用 masked BCE:

$$\mathcal{L}_{\text{sleep}} = \frac{\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{sl}} \mathcal{L}_{\text{BCE}}(p_{b,t}^{\text{sl}}, y_{b,t}^{\text{sl}})}{\max(\sum_{b=1}^B \sum_{t=1}^T m_{b,t}^{\text{sl}}, 1)}$$

引入该辅助任务的原因是, onset 和 wakeup 不只是两个孤立点, 而是睡眠区间的边界.让模型同时学习“当前是否处于睡眠状态”, 可以帮助它获得更完整的时序结构理解.

VII.4.4 Boundary Consistency Loss

为了让 sleep 通道和事件通道之间保持一致, 本项目还设计了边界一致性损失.直观地说, 如果相邻两个 step 的 sleep 概率发生明显变化, 那么附近应该存在较高的 onset 或 wakeup 概率.否则, 模型可能会预测出突变的睡眠状态, 但事件通道没有对应边界.

首先定义 sleep 概率变化量:

$$\Delta p_{b,t}^{\text{sl}} = |p_{b,t}^{\text{sl}} - p_{b,t-1}^{\text{sl}}|$$

再定义事件边界概率:

$$p_{b,t}^{\text{bd}} = \max(p_{b,t}^{\text{on}}, p_{b,t}^{\text{wu}})$$

如果 sleep 状态变化量大于事件边界概率, 则说明二者不一致, 需要惩罚.因此边界一致性损失定义为:

$$\mathcal{L}_{\text{boundary}} = \frac{\sum_{b=1}^B \sum_{t=2}^T m_{b,t}^{\text{valid}} \text{ReLU}(\Delta p_{b,t}^{\text{sl}} - p_{b,t}^{\text{bd}})}{\max(\sum_{b=1}^B \sum_{t=2}^T m_{b,t}^{\text{valid}}, 1)}$$

其中: $\text{ReLU}(x) = \max(x, 0)$

这个 loss 的作用是: 当 sleep 概率变化很大, 但 onset/wakeup 概率没有相应升高时, 模型会受到惩罚; 如果事件通道已经给出了较高边界概率, 则不会额外惩罚.

VII.4.5 总损失函数

最终训练使用的总损失为:

$$\mathcal{L} = \lambda_{\text{event}} \mathcal{L}_{\text{event}} + \lambda_{\text{sleep}} \mathcal{L}_{\text{sleep}} + \lambda_{\text{boundary}} \mathcal{L}_{\text{boundary}} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}}$$

在主要实验中, rank loss 没有启用, 即:

$$\lambda_{\text{rank}} = 0$$

因此实际主要使用：

$$\mathcal{L} = \lambda_{\text{event}} \mathcal{L}_{\text{event}} + \lambda_{\text{sleep}} \mathcal{L}_{\text{sleep}} + \lambda_{\text{boundary}} \mathcal{L}_{\text{boundary}}$$

event_plus 实验中的权重为：

yaml

```
1 event_weight: 1.5
2 sleep_weight: 0.35
3 boundary_weight: 0.1
4 rank_weight: 0.0
```

总体目的是优化 onset/wakeup 事件检测，同时利用 sleep 区间作为辅助监督，并用边界一致性约束增强事件概率和睡眠状态变化之间的逻辑关系。

VII.4.6 损失函数总结

整体来看，本项目的 loss function 设计服务于三个目标：

- (1) 通过高斯软标签缓解事件点过于稀疏的问题。
- (2) 通过 focal loss 和更高的 event_weight 让模型关注难样本和稀疏事件。
- (3) 通过 sleep_loss 和 boundary_loss 引入睡眠区间结构，使模型不仅学习孤立事件点，也学习睡眠状态的连续变化。

实验中，加入 focal loss 和事件权重后，本地 OOF 分数从约 0.077 提升到约 0.081，说明该损失设计确实增强了模型对事件区域的关注。不过该提升没有完全转化为 Private Score 提升，也说明更强的事件敏感性可能同时带来更多 false positive，需要与后处理阈值共同调节。

下面这段可以直接放到 proj-2.typ 里的“后处理方法”小节。它对应你代码中的 src/postprocess.py。

VII.5 后处理方法

模型的直接输出不是最终提交文件，而是每个 series_id、每个 step 上的连续概率序列：

$$p_t^{\text{on}}, p_t^{\text{wu}}, p_t^{\text{sl}}, p_t^{\text{inv}}$$

其中 p_t^{on} 表示第 t 个 step 为 onset 的概率， p_t^{wu} 表示第 t 个 step 为 wakeup 的概率， p_t^{sl} 表示当前 step 处于睡眠区间的概率， p_t^{inv} 表示当前 step 可能属于无效片段的概率。

但是 Kaggle 要求提交的是离散事件列表：

text

```
1 row_id, series_id, step, event, score
```

因此，需要将连续概率序列转换为若干个带置信度的事件点。这个过程就是后处理。后处理对本任务非常重要，因为评价指标是基于事件匹配的 AP，而不是逐 step 分类准确率。

VII.5.1 概率平滑

模型输出的概率序列可能存在局部噪声。例如，在真实 onset 附近，模型可能不是输出一个平滑峰值，而是在相邻 step 上出现多个小尖峰。为了让事件峰值更加稳定，首先对每个事件通道的概率进行滑动平均平滑。

对于事件类型 $e \in \{\text{onset}, \text{wakeup}\}$ ，记模型原始概率为 p_t^e

平滑后的概率记为 $\tilde{p}_t^e$

若平滑窗口大小为 w ，则可以写成：

$$\tilde{p}_t^e = \frac{1}{|W_t|} \sum_{u \in W_t} p_u^e$$

其中 W_t 表示以 t 为中心、长度约为 w 的局部窗口.代码中使用 `scipy.ndimage.uniform_filter1d` 实现该操作.

主要实验中的配置为:

yaml

```
1 smooth_window_steps: 24
```

由于 1 step = 5 秒, 24 step 约等于 2 分钟.这个尺度能够消除很短的概率噪声, 同时不会过度抹平真正的事件峰值.

VII.5.2 峰值检测

平滑后, 对 onset 和 wakeup 两个事件通道分别进行峰值检测.也就是说, 对于每个 `series_id` 和每种事件类型 e , 寻找满足局部最大条件的 step:

$$\tilde{p}_t^e \geq \tilde{p}_{t-1}^e \quad \wedge \quad \tilde{p}_t^e \geq \tilde{p}_{t+1}^e$$

实际代码中使用 `scipy.signal.find_peaks`, 并结合高度阈值和最小峰值间距筛选候选事件.

峰值检测的意义是: 模型概率序列可以看作事件发生可能性的时间分布, 而真正提交时只需要概率曲线中的若干局部最大点.

VII.5.3 最小峰值间距限制

如果不限制峰值间距, 模型可能会在同一个真实事件附近生成很多非常接近的候选点.由于评价指标中每个真实事件最多只能匹配一次, 多余候选会变成 false positive, 从而降低 AP.

因此, 代码中设置最小峰值间距:

yaml

```
1 peak_distance_steps: 180
```

同一条 `series_id`、同一事件类型下, 两个候选峰值之间至少相隔约 15 分钟.这个设置可以减少重复预测, 使候选事件在时间轴上更加分散.

VII.5.4 阈值过滤

峰值检测时还会使用分数阈值过滤低置信度候选.对于候选 step t , 如果平滑概率低于阈值, 则不会作为正式候选事件:

$$\tilde{p}_t^e < \theta \Rightarrow \text{discard}$$

主要配置为:

yaml

```
1 score_threshold: 0.0001
```

该阈值看起来很低, 这是因为本任务的 AP 指标对召回率比较敏感.如果阈值过高, 模型可能漏掉一些低置信度但位置正确的事件; 如果阈值过低, 则会产生更多 false positive.因此这里采用较低阈值, 再通过峰值间距和最大候选数控制噪声.

VII.5.5 候选事件置信度计算

候选事件的最终 score 并不是简单等于峰值高度, 而是综合考虑三个因素:

- (1) 峰值处的平滑概率.
- (2) 峰值附近原始概率的局部平均质量.

(3) 无效片段概率带来的惩罚.

对于事件类型 e , 候选 step t 的分数定义为:

$$s_{e(t)} = \alpha \tilde{p}_t^e + (1 - \alpha) \text{Mass}_{e(t)} - \beta \text{Invalid}(t)$$

其中:

$$\text{Mass}_{e(t)} = \frac{1}{|U_t|} \sum_{u \in U_t} p_u^e$$

表示候选点附近原始概率的平均值.代码中 U_t 大约取 $[t - 12, t + 12]$ 的局部区域, 即候选点前后各 12 step, 约 2 分钟范围.

无效片段惩罚项为:

$$\text{Invalid}(t) = \frac{1}{|U_t|} \sum_{u \in U_t} p_u^{\text{inv}}$$

如果模型认为该区域可能是无效片段, 则候选事件分数会被降低.

主要配置为:

```
1 peak_weight: 0.85
2 mass_weight: 0.15
3 invalid_penalty: 0.05
```

yaml

最终分数会被裁剪到 $[0, 1]$:

$$\text{score}_e(t) = \text{clip}(s_e(t), 0, 1)$$

这样做的好处是, 单个尖锐峰值和宽而稳定的概率区域都能被考虑到.峰值高说明模型在某个 step 很确定, 局部 mass 高说明该区域整体都接近事件, 而 invalid penalty 可以减少低质量片段造成的误报.

VII.5.6 最大候选数限制

对于每条 series_id 和每种事件类型, 后处理只保留分数最高的若干个候选事件.主要配置为:

```
1 max_events_per_series_event: 200
```

yaml

这个限制可以避免某些长序列生成过多低质量预测, 从而控制 false positive 数量.

VII.5.7 无峰值时的 fallback

在极少数情况下, 如果某条序列的某个事件通道没有检测到任何峰值, 但该序列长度大于 0, 代码会选择平滑概率最大的 step 作为 fallback 候选:

$$t^* = \arg \max_t \tilde{p}_t^e$$

这样可以保证每条序列至少有基本候选, 避免因为阈值或概率异常导致完全没有预测结果.

VII.5.8 Pair Filter 实验

除了基础峰值后处理, 本项目还尝试了 pair_candidates, 即事件配对过滤.睡眠事件具有天然结构: 一次睡眠通常由一个 onset 和之后的一个 wakeup 组成.因此可以根据睡眠时长约束对候选事件进行配对.

对于一个候选 onset step t_{on} 和候选 wakeup step t_{wu} , 只有满足以下条件才认为它们可能构成一对:

$$t_{\text{wu}} > t_{\text{on}} + d_{\text{min}}$$

$$t_{\text{wu}} < t_{\text{on}} + d_{\text{max}}$$

其中代码中的默认约束为:

```
1 min_sleep_steps: 360
2 max_sleep_steps: 11520
```

yaml

换算成时间也就是说, 一段有效睡眠时长需要大于约 30 分钟, 小于约 16 小时.

配对时还会利用 onset 分数、wakeup 分数、两者之间的平均 sleep 概率和 invalid 概率计算 pair score:

$$s_{\text{pair}} = \frac{\text{score}_{\text{on}} + \text{score}_{\text{wu}}}{2} + \lambda_{\text{sl}} \text{mean}(p^{\text{sl}}) - \lambda_{\text{inv}} \text{mean}(p^{\text{inv}})$$

其中 $\text{mean}(p^{\text{sl}})$ 和 $\text{mean}(p^{\text{inv}})$ 分别在 onset 到 wakeup 之间的区间上计算. 直观地说, 如果两个事件之间模型认为大部分时间处于睡眠状态, 那么这对事件更可信; 如果中间无效概率较高, 则降低得分.

不过在实际实验中, Pair Filter 并没有稳定提升分数. 可能原因是测试集中存在不规则睡眠片段, 过强的配对先验会误删一些真实事件. 因此最终主力提交仍以基础峰值检测和阈值过滤为主.

VII.5.9 Submission 生成

完成候选筛选后, 所有候选事件会被整理为 Kaggle 要求的格式:

```
1 row_id, series_id, step, event, score
```

text

其中 row_id 按顺序重新编号, step 转换为整数, score 裁剪到 $[0, 1]$, event 必须属于:

{onset, wakeup}

如果后处理结果为空, 代码会为每条序列生成低分 fallback 行, 保证提交文件格式合法.

可以, 这部分可以按“先放结果表, 再逐项分析, 最后总结反思”的结构写. 下面这段可以直接放进报告.

VII.6 实验结果分析

本项目进行了多组实验, 包括基础模型、扩大训练规模、模型融合、事件增强损失、后处理调试以及内存优化实验. 实验结果如下表所示:

实验设置	OOF Local	Private	Public	结论
Small baseline	0.0409	0.411	0.363	验证流程跑通.
Medium+ 2-fold	0.0777	0.605	0.534	最佳稳定版本.
Large 2-fold	0.0700	0.552	0.484	更大模型未带来提升.
Medium+ 3-model ensemble	-	0.597	0.517	简单融合收益有限.

Event-focused + focal loss	0.0814	0.597	0.543	本地提升明显.
Debug: pair filter	0.0380	-	-	后处理策略不稳定, 未提交.
Debug: boundary mix	0.0757	-	-	结构改良未显著提升.
Debug: fullish lazy windows	0.0559	-	-	实现和内存问题, 未提交.
Debug: event plus w2	0.0814	-	-	参数搜索实验, 未提交.

VII.6.1 Small baseline: 验证完整流程

Small baseline 的本地 OOF 分数为 0.0409, Private Score 为 0.411, Public Score 为 0.363.这一版本的主要意义不是追求高分, 而是验证整个 pipeline 是否正确, 包括数据读取、特征工程、窗口切分、模型训练、推理、后处理和 Kaggle 提交.

从结果看, baseline 已经能够产生有效预测, 说明模型确实学习到了一些睡眠事件相关模式.但分数较低, 说明训练数据覆盖、模型容量、训练轮数和后处理策略都还不够充分.

VII.6.2 Medium+ 2-fold: 最佳稳定版本

Medium+ 2-fold 的本地 OOF 分数为 0.0777, Private Score 为 0.605, Public Score 为 0.534, 是本项目中最稳定、综合表现最好的版本.

相比 small baseline, 该版本扩大了训练 series 数量和训练强度, 并使用 2-fold 验证.它在本地 OOF、Public 和 Private 上都有明显提升, 说明更充分的数据覆盖和更稳定的验证划分对该任务非常重要.

最终选择该版本作为主力提交, 是因为它不仅 Public 分数较高, 而且 Private 分数最高, 说明泛化能力相对更可靠.

VII.6.3 Large 2-fold: 模型变大不一定更好

Large 2-fold 的 OOF 分数为 0.0700, Private Score 为 0.552, Public Score 为 0.484, 相比 Medium+ 2-fold 反而下降.

这说明在本任务中, 模型规模扩大并没有直接带来提升.可能原因包括:

- (1) 模型容量变大后更容易过拟合训练子集.
- (2) Kaggle 训练资源有限, 较大模型可能没有充分收敛.
- (3) batch size 受显存限制较小, 训练噪声较大.
- (4) 后处理参数仍沿用 medium 配置, 未必适合 large 模型输出的概率分布.

因此, 模型复杂度需要与数据量、训练预算和后处理策略匹配, 而不是简单地增大网络规模.

VII.6.4 Medium+ 3-model ensemble: 简单融合收益有限

Medium+ 3-model ensemble 的 Private Score 为 0.597, Public Score 为 0.517, 没有超过单个 Medium+ 2-fold 模型.

模型融合通常要求不同模型之间具有互补性.例如, 不同模型结构、不同窗口长度、不同 loss function、不同随机种子或不同特征组合, 都可能带来互补信息.但本次融合主要基于相似配置的模型, 它们的预测分布和错误模式比较接近, 因此简单平均或融合不能显著提升结果.

这说明 ensemble 并不是无条件有效.只有当多个模型犯错方式不同, 融合才更可能提高鲁棒性.

VII.6.5 Event-focused + focal loss: 本地提升明显但 Private 未提升

Event-focused + focal loss 的 OOF Local 达到 0.0814, 是所有正式实验中最高的本地分数. 它的 Public Score 为 0.543, 高于 Medium+ 2-fold 的 0.534, 但 Private Score 为 0.597, 略低于最佳版本的 0.605.

这一结果说明 event-focused sampling 和 focal loss 确实增强了模型对稀疏事件的关注, 使模型更容易在事件附近产生概率峰值. 但它也可能带来更多 false positive, 尤其是在 Private 测试集分布与本地验证集不完全一致时, 过强的事件敏感性反而会降低最终 AP.

因此, 本地 OOF 提升并不一定完全等价于 Private Score 提升. 对于 Kaggle 任务, 模型选择需要综合考虑本地验证、Public Score 和方法稳定性.

VII.6.6 Pair filter: 后处理先验过强

Debug: pair filter 的本地 OOF 只有 0.0380, 明显低于主力模型, 因此没有提交.

Pair filter 的设计初衷是利用睡眠事件成对出现的先验: 一次睡眠通常由一个 onset 和之后的一个 wakeup 构成, 因此可以用合理睡眠时长约束过滤候选事件. 然而实际结果较差, 说明这个后处理策略不够稳定.

可能原因是, 真实数据中存在不规则睡眠、缺失标注、短睡眠片段或传感器噪声. 如果强制进行事件配对, 可能会误删一些本来可以匹配真实事件的候选点, 导致 recall 下降.

VII.6.7 Boundary mix: 结构改良未显著提升

Debug: boundary mix 的 OOF Local 为 0.0757, 接近但没有超过 Medium+ 2-fold 的 0.0777.

该实验尝试将 sleep 概率变化与 onset/wakeup 事件概率进一步结合, 希望利用睡眠状态边界帮助事件定位. 但结果显示, 这种结构改良没有带来显著提升.

可能原因是模型原本已经通过 boundary_loss 学到了一部分边界一致性, 再额外进行 boundary mix 可能带来冗余, 甚至干扰事件通道本身的概率分布.

VII.6.8 Fullish lazy windows: 工程优化不能改变数据语义

Debug: fullish lazy windows 的 OOF Local 为 0.0559, 明显低于主力版本, 因此没有提交.

该实验的目标是解决 Kaggle 内存不足问题. 原始版本会提前计算并保存较多窗口特征, 占用内存较大; lazy windows 希望在训练时按需构建窗口, 从而降低内存压力.

但是初版实现存在一个关键问题: rolling feature 是在单个窗口内部计算的, 而原始实现是在整条 series 上计算 rolling feature 后再切窗口. 这样会改变窗口边界附近的统计特征, 使特征语义发生变化, 最终导致分数下降.

这个实验说明, 对于时间序列任务, 工程优化不能只看内存和速度, 还必须保证特征计算逻辑与原始方法一致.

VII.6.9 Event plus w2: 参数搜索实验

Debug: event plus w2 的 OOF Local 同样为 0.0814, 属于 event plus 方向的参数搜索实验. 由于它主要用于验证参数变化, 没有进一步提交到 Kaggle.

该结果说明 event-focused 和 focal loss 方向有一定潜力, 但还需要更系统地调节后处理阈值、峰值距离、候选数量和事件权重, 才能判断它是否真正优于稳定的 Medium+ 2-fold.

VII.7 实验反思

VII.7.1 本地 OOF 与 Kaggle 分数并不完全一致

从实验结果可以看出, 本地 OOF 分数和 Kaggle Private Score 并不总是严格一致.例如 Event-focused + focal loss 的 OOF Local 最高, Public Score 也较高, 但 Private Score 没有超过 Medium+ 2-fold.

这说明本地验证集与隐藏测试集之间仍然存在分布差异.可能原因包括:

- (1) 训练和验证只使用 2-fold, 验证稳定性有限.
- (2) 不同 series_id 的睡眠模式、噪声水平和佩戴情况差异较大.
- (3) 后处理参数对不同数据分布比较敏感.
- (4) Public Score 只反映测试集的一部分, 不能完全代表最终表现.

因此, 模型选择不能只看单一指标, 而要综合考虑 OOF、Public、Private 以及实验逻辑.

VII.7.2 模型不是越大越好

Large 2-fold 的结果低于 Medium+ 2-fold, 说明更大的模型并不一定带来更好效果.在训练数据有限、训练时间有限、batch size 较小的情况下, 大模型可能更容易过拟合, 也可能因为训练不足而表现不稳定.

对于实际 AI 项目, 模型规模需要和数据量、计算资源、训练轮数、正则化策略共同设计.一个中等规模但训练充分、后处理稳定的模型, 可能比一个更大但调参不足的模型更可靠.

VII.7.3 后处理对最终分数影响很大

本任务的评价对象是离散事件, 而不是逐 step 概率.因此后处理直接决定模型概率如何转换为提交结果.pair filter 实验分数较低说明, 后处理先验如果设计不当, 可能会严重影响 recall.

后处理中的平滑窗口、峰值间距、阈值、候选数量和 score 计算方式, 都需要与模型输出分布匹配.模型本身只是系统的一部分, 后处理同样是最终性能的重要来源.

VII.7.4 工程实现也是模型效果的一部分

fullish lazy windows 实验说明, 工程优化可能改变模型输入特征的语义.尤其在时间序列任务中, rolling feature 依赖上下文, 如果从整条序列计算改成从窗口内部计算, 边界位置的统计量就会不同.

因此, 优化内存和速度时必须同时检查特征一致性.一个看似等价的实现, 如果改变了数据处理方式, 最终可能导致明显性能下降.

VII.7.5 失败实验同样有价值

虽然 large、ensemble、pair filter、boundary mix 和 lazy windows 没有取得更高分数, 但这些实验帮助我理解了不同改动的影响:

- (1) large 实验证明模型容量不是唯一因素.
- (2) ensemble 实验证明融合需要模型差异性.
- (3) pair filter 实验证明强先验可能降低召回.
- (4) boundary mix 实验证明结构改动需要实际验证.
- (5) lazy windows 实验证明工程优化不能改变特征语义.

这些失败结果为后续改进提供了方向, 也让整个项目不只是跑出一个分数, 而是形成了比较完整的实验分析过程.

总体来看, Medium+ 2-fold 是本项目最稳定的版本, 取得了 Private = 0.605、Public = 0.534 的最好综合结果; Event-focused + focal loss 在本地和 Public 上表现更强, 说明事件增强方

向有潜力,但仍需要进一步控制 false positive.通过这些实验可以看到,时间序列事件检测任务需要同时平衡模型结构、损失函数、采样策略、后处理和工程实现,任何单独部分的改动都可能影响最终结果.

VIII 总结

本项目完成了一个较完整的深度学习时间序列事件检测系统.系统从 Kaggle 原始传感器数据出发,经过特征工程、软标签构造、窗口采样、Sleep-Mamba-UNet 训练、概率后处理和提交文件生成,最终获得了较好的 Kaggle 分数:

text

```
1 Private Score: 0.605
2 Public Score: 0.534
```

本项目让我完整经历了真实 AI 项目的闭环:理解任务、设计表示、选择模型、处理类别不平衡、调试工程问题、提交结果并分析失败原因.通过这次实验,我对时间序列深度学习、事件检测、Kaggle 实验流程和模型评估有了更具体的认识.